スマート駐車管理システム

ビラ（小規模集合住宅）向けAI駐車推薦 — プログラミングスキル紹介

Python / Django | YOLOv8 + OCR | Gaussian Process | React + i18next | Docker / AWS EC2

このプロジェクトを通じて成長したこと

問題定義 → アルゴリズム設計 → 実装 → デバッグ → 検証

フルスタック開発をチームで設計・実装し、コンテスト後は一人で改善しながら「プログラマーとしての思考法」を体得

→ この経験を活かし、実際に動くシステムを設計・運用できるエンジニアを目指しています

GitHub URL : https://github.com/DohyunKang

制作過程での工夫と成長

実際にぶつかった問題 → 自ら考え → 解決 → 学んだこと

① OCR認識率の低下

カメラ角度・照明によりナンバープレート誤認識
単純な文字列比較ではDB照合が不可能

工夫: difflib類似度 → ファジーマッチング（70%以上）で補完

✅ 学び: アルゴリズム選択が結果を左右することを体得

② 強化学習のデータ不足

実際の駐車データが少なく強化学習が収束しない
予測精度が実用レベルに達しなかった

工夫: Gaussian Process へ移行 — ベイズ推論 + 不確実性定量化

✅ 学び:「動くコード」より「正しい設計」が先だと体得

③ API連携エラー & デバッグ

Google Maps・Django CSRFなど外部API連携時
コンソールエラーだけでは原因特定が困難

工夫: Networkタブ・console.logで段階的追跡 → 根本原因を特定

✅ 学び: エラーを再現し範囲を絞るデバッグ思考法を体得

④ 一人でフルスタック設計・実装

DB設計からAI統合・デプロイまで全工程を
チームリーダーとして一人で設計・実装

工夫: コンポーネント責任を明確に分離し、I/F先行定義を採用

✅ 学び: システム全体を見渡す視野を獲得

コア技術の実装 — Before / After

単に実装しただけでなく、なぜこの方法かを理解して選択しました

🔧 OCR認識エラー → ファジーマッチングで解決

【問題】OCRが認識した文字列にノイズ・誤認識が多発

```
# Before
ocr_text = reader.readtext(img)
if ocr_text == db_plate:
    match = True
```

```
# After: 類似度ベース照合
from difflib import get_close_matches
def fuzzy_match_plate(ocr_text, db_plates):
    matches = get_close_matches(
        ocr_text, db_plates,
        n=1, cutoff=0.7)
    return matches[0] if matches else None
```

✓ 認識精度向上 ✓ ソフトウェアのみで解決

🤖 強化学習 → Gaussian Processへ改善

【問題】強化学習はデータ収集コスト・収束が遅い

```
# Before: 強化学習ベース
# 大量データ・長期訓練が必要
# データ不足環境では精度が低い
```

```
# After: Gaussian Process回帰
from sklearn.gaussian_process import (
    GaussianProcessRegressor)
gp = GaussianProcessRegressor(
    kernel=RBF(length_scale=1.0),
    n_restarts_optimizer=5)
gp.fit(X_train, y_train)
y_pred, sigma = gp.predict(
    X_test, return_std=True)
```

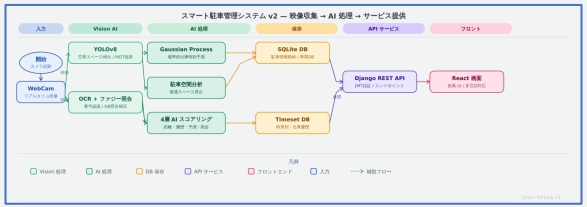
✓ 少データで高精度 ✓ 不確実性定量化 ✓ 説明可能

スクリーンショット・システム設計図

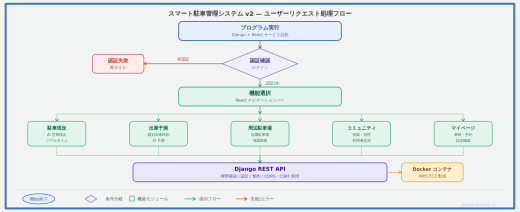
実際の動作画面と設計構造



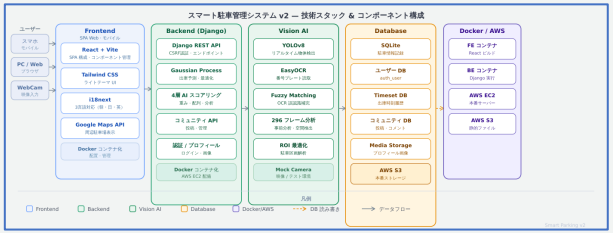
📷 YOLOv8 リアルタイム認識 + OCR（実稼働画面）



🔄 データフロー図



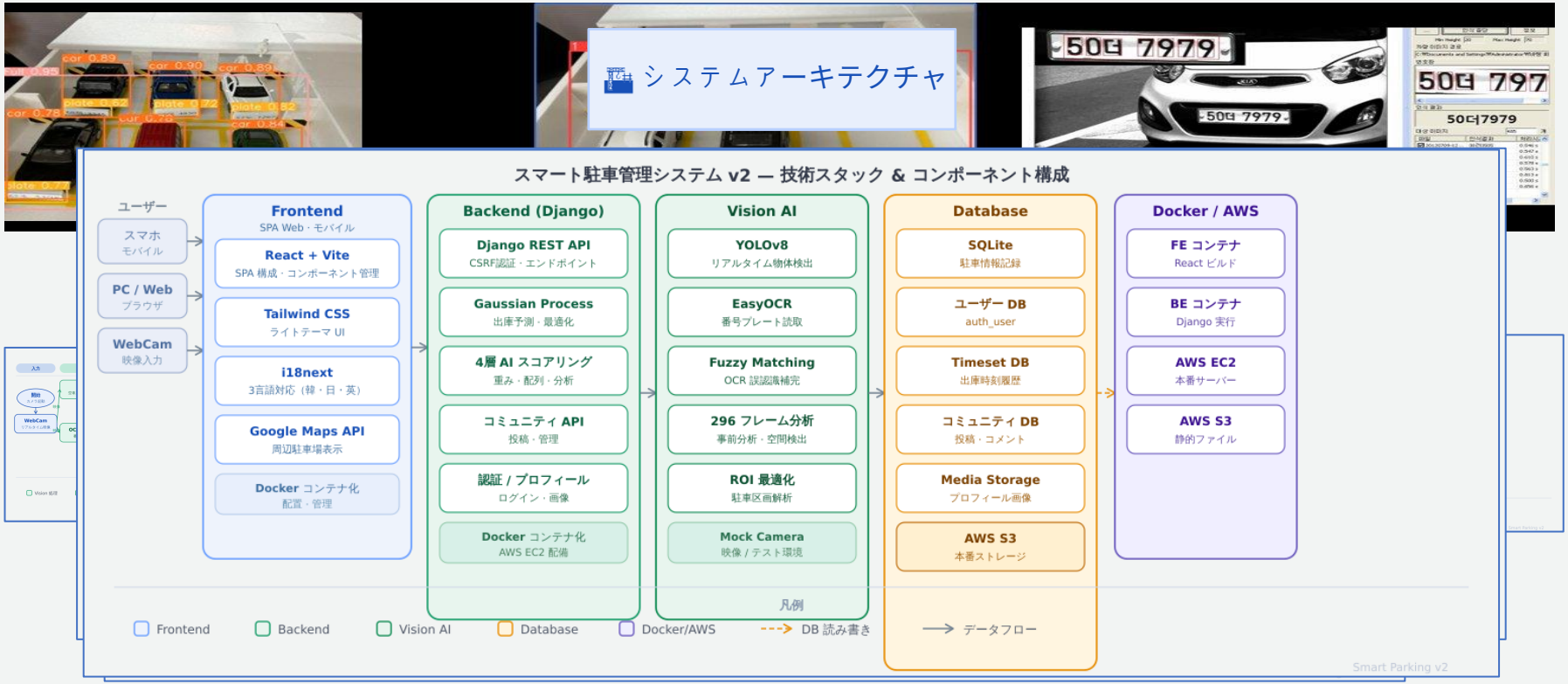
⚙ 機能フロー図



🏗 システムアーキテクチャ

スクリーンショット・システム設計図

実際の動作画面と設計構造



コンテスト後の自主改善 — 追加開発レポート

受賞で終わらせず、一人で実際に動くサービスへ進化させた

📦 コンテストで実装・体得したこと

映像 — **YOLOv8リアルタイム映像処理**
毎フレーム物体検出・追跡・状態更新

AI — **Gaussian Process 予測AI**
過去データで将来行動を予測・不確実性計算

同期 — **リアルタイムポーリング & 同期**
1.5秒間隔の状態同期・マルチクライアント処理

循環 — **デバッグ・テスト・検証サイクル**
問題再現 → 範囲絞り → 修正 → 再検証

Docker — **Dockerコンテナ設計**
環境独立ビルド・AWS EC2即時デプロイ可能

→ 改善

🚀 コンテスト後に一人で追加・改善したこと

UI — **ダークテーマへの全面リデザイン**
「駐車の達人」として統一感ある実プロダクトUIに刷新

認証 — **会員登録・ソーシャルログイン**
Google・Kakao連携と登録国による言語自動設定

会話 — **リアルタイムチャット機能**
ドライバー同士が即連絡・自由掲示板での投稿も追加

管理 — **マイページ・お知らせ管理**
プロフィール・出庫時間の設定と管理者お知らせ機能

多言語 — **3言語対応・レスポンシブ対応**
韓・英・日の3言語対応とスマホでも使えるレスポンシブUI

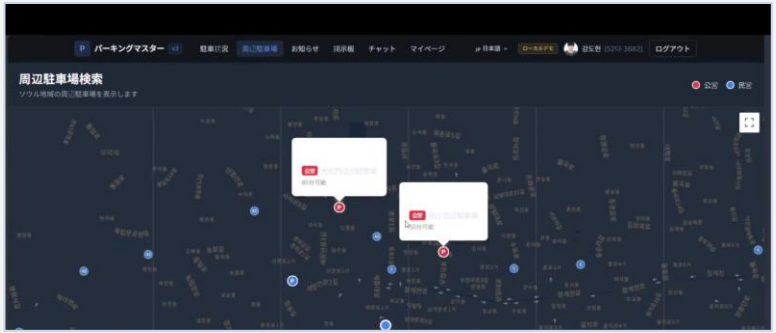
改善版の実画面 —「駐車の達人」

コンテスト後に一人で作り込んだ実際の動作画面

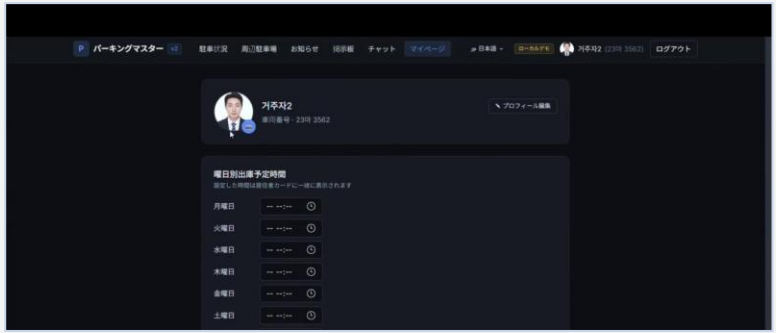
メイン画面 — AI推薦・リアルタイム空き状況



周辺駐車場検索 — Google Maps連携



マイページ — プロフィール・出庫時間設定



レスポンシブ対応 — スマホでも利用可能



成果とまとめ

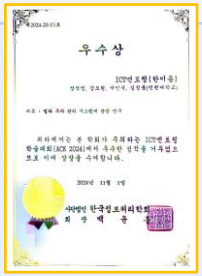
このプロジェクトが残したもの

ビラ駐車場という実際の課題に直接取り組み、OCRエラー・データ不足・APIデバッグなど数多くの難題を
アルゴリズム設計と数学的思考で自ら解決しながら、
「問題定義 → 設計 → 実装 → デバッグ → 検証」の開発サイクルを体得しました。
その結実として、ACK 2024学術大会 論文優秀賞と HANIUM ICT メンタリング公募展 入選を受賞しました。
また、コンテスト終了後も一人でフロントエンドのリファクタリングと AI ロジックの改善を続け、単なる受賞にとどまらず、実際に動作するシステムへと進化させました。

500+@	3言語	4層AI	2冠
フレーム 事前分析	韓・日・英 国際化	統合 スコアリング	ACK優秀賞 + ICT入選

受賞証明書

ACK 2024学術大会 論文優秀賞



HANIUM ICT メンタリング公募展 入選

